

Assignment 1:

Use Case Chosen

Software Developer Assistant

Problem Description

Junior software developers often struggle to understand JavaScript runtime errors. They usually receive vague explanations or fixes without understanding the root cause, which slows down their learning and productivity.

AI Solution Overview

The AI will act as a senior software developer and mentor. It will analyze JavaScript errors, explain the root cause in simple language, provide corrected code, and suggest best practices to avoid similar issues in the future.

Initial Prompt (Version 1)

You are a software developer.
Help me fix JavaScript errors and explain them.

Improved Prompt (Version 2)

You are a senior JavaScript engineer and mentor.

Your task:

1. Analyze the given JavaScript error.
2. Explain the root cause in simple terms suitable for junior developers.
3. Provide the corrected code.
4. Explain why the fix works.
5. Suggest one best practice to avoid this error in the future.

Rules:

- Use clear step-by-step explanations.
- Avoid complex jargon.
- Format your answer using headings and bullet points.

Explanation of Improvement

The improved prompt assigns a clear expert role to the AI and provides structured instructions. This resulted in a clearer, more educational, and well-organized response that better supports junior developers.

AI Chat Link

<https://chatgpt.com/share/6965560d-6c84-8013-819e-ccd7adac7bc2>

Assignment 2:

1. Self-Critique Prompting

Self-critique prompting is a technique where the AI is instructed to review and evaluate its own response before presenting the final output. The model first generates an initial answer, then analyzes it for clarity, accuracy, or missing information, and finally produces an improved version.

This technique improves AI responses by reducing errors, increasing completeness, and enhancing clarity. By forcing the model to reflect on its output, the final response is usually more accurate and structured.

This technique is especially useful in technical explanations, report writing, and decision-making tasks where precision is important.

2. Output Validation and Constraint Prompting

Output validation and constraint prompting involves explicitly defining strict rules for the AI's output, such as word limits, formatting requirements, or content restrictions. For example, the AI can be instructed to respond using bullet points only, limit the answer to a specific number of steps, or avoid certain terms.

This technique improves responses by making them more predictable, readable, and aligned with user expectations. It also reduces irrelevant or overly verbose answers.

This approach is useful in professional documentation, customer support responses, and educational content where consistency and clarity are required.

3. Persona Switching (Dynamic Role Prompting)

Persona switching is an advanced form of role prompting where the AI is instructed to change its role dynamically during the same interaction. For example, the AI may first respond as a software engineer and then as a technical reviewer or educator.

This technique improves AI responses by offering multiple perspectives and deeper analysis within a single interaction. It helps the model provide richer, more balanced outputs.

Persona switching is useful in problem analysis, design reviews, and learning scenarios where explanation and evaluation are both needed.

Assignment 3:

With the rapid advancement of Generative Artificial Intelligence, new methods have emerged to improve how users interact with AI models. Two important concepts in this field are Prompt Engineering and Context Engineering. Although they are closely related, they serve different purposes and are used in different ways.

Prompt Engineering

Prompt Engineering refers to the practice of designing and optimizing the instructions given to an AI model to obtain the best possible response. It focuses on how the user phrases questions, defines tasks, assigns roles, and specifies output formats.

Examples of prompt engineering techniques include role prompting, step-by-step instructions, few-shot examples, and output constraints. The main goal of prompt engineering is to guide the model toward producing accurate, clear, and relevant responses within a single interaction.

Context Engineering

Context Engineering involves managing and supplying background information that the AI uses across multiple interactions. Instead of focusing on a single prompt, it focuses on maintaining useful context such as previous conversations, user preferences, system instructions, or external knowledge.

Context engineering ensures consistency, personalization, and continuity in AI responses. It is especially important in long conversations, chatbots, and AI systems that need to remember past interactions or adapt to specific user needs.

Key Differences Between Prompt Engineering and Context Engineering

Aspect	Prompt Engineering	Context Engineering
Focus	Single prompt design	Long-term interaction and memory
Scope	Instructions and formatting	Background information and state
Usage	Immediate task execution	Consistency across conversations
Example	Asking AI to act as a senior developer	Remembering user skill level over time

Real-World Examples

An example of prompt engineering is instructing an AI to explain a programming error step by step using simple language. On the other hand, context engineering can be seen when an AI remembers that a user is a beginner programmer and continuously adjusts explanations accordingly throughout the conversation.

Conclusion

In conclusion, Prompt Engineering and Context Engineering are complementary techniques. Prompt engineering focuses on crafting effective instructions for immediate tasks, while context engineering focuses on maintaining relevant information over time. Using both techniques together results in more accurate, personalized, and efficient AI systems.